

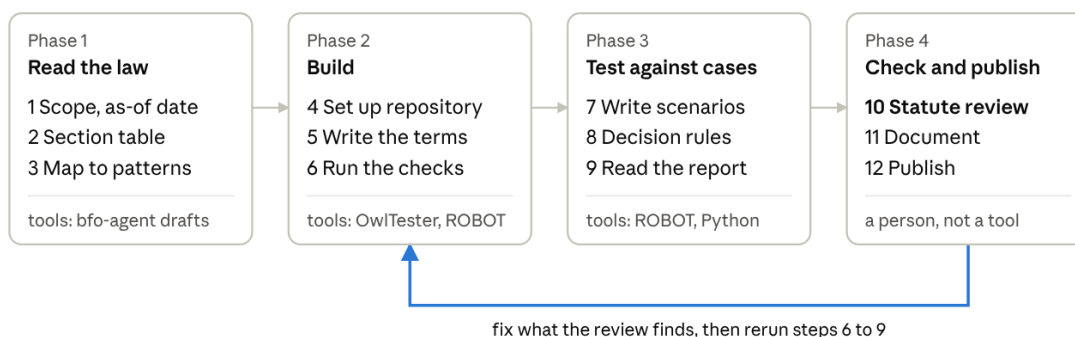
# How to Build a Statute Ontology: The TX-DNR Method

Oct 9, 2026 · @David Koepsell

This guide walks a person through building a BFO-grounded ontology of a statute in twelve steps, using the Texas DNR ontology (TX-DNR) as the worked example. Two of our own tools speed it up: **bfo-agent** drafts a first-pass ontology from the statute's text, and **OwlTester** (the FOL-BFO-OWL Tester) checks expressions and finished files against BFO. Neither replaces the human steps that keep the ontology faithful to the law: the section table, the scenarios, and the independent statutory review.

## At a glance

Twelve steps in four phases; the statutory review loops back to the build



the method · 4 phases, 12 steps, 1 review loop

Phases 1 to 3 build an ontology that passes every automated check; step 10 tests whether it is faithful to the statute, and its findings send you back to the build.

## What you need

Everything is free. Install the tools once; the files come from the Foundry and BFO repositories.

| Item                                  | What it is for                                                                                                                                                              | Where to get it                                                                                                                                    |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>bfo-agent</b>                      | Drafting: reads a source text and proposes OWL classes grounded in BFO, under a closed vocabulary and anti-pattern rules, then passes each draft through the OwlTester gate | Our bfo-agent repositories<br><a href="#">BFO-Agent</a><br><a href="https://bfo-agent.davidkoepsell.com/">https://bfo-agent.davidkoepsell.com/</a> |
| <b>OwlTester</b> (FOL-BFO-OWL Tester) | Checking: tests single FOL expressions for correct BFO                                                                                                                      | <a href="https://ontology.davidkoepsell.com">ontology.davidkoepsell.com</a>                                                                        |

| Item                                        | What it is for                                                                                                        | Where to get it                                                                 |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
|                                             | usage, and analyzes uploaded OWL files for consistency, coherence, BFO category conflicts, with a Prover9 cross-check |                                                                                 |
| Protégé 5                                   | Writing and revising classes, relations, and definitions by hand, and running the reasoner interactively              | <a href="http://protege.stanford.edu">protege.stanford.edu</a>                  |
| Java 11 or later                            | Runs Protégé and ROBOT                                                                                                | Your operating system's package manager                                         |
| ROBOT                                       | Command-line merging, reasoning, QC checks, and releases                                                              | <a href="http://robot.obolibrary.org">robot.obolibrary.org</a>                  |
| Python 3 with rdflib                        | Running the decision rules and the plain-language report                                                              | <code>pip install rdflib</code>                                                 |
| BFO 2020 core (OWL)                         | The upper ontology every class must descend from                                                                      | <a href="http://bfo-2020-repository">BFO-2020 repository</a>                    |
| LOF-ODP-001 to 004 base files               | Legal act, content, document, role, position, and Minimum Legal Chain                                                 | The four lof-odp-00N repositories in the Foundry organization                   |
| The statute, current text and history notes | The source of every term                                                                                              | The state's official code site, plus an annotated mirror for history notes      |
| A second reader                             | Independent check of every legal claim (step 10)                                                                      | A colleague, a student, or a separate agent session that has not seen your work |
| A text editor and GitHub account            | Editing Turtle files directly, publishing                                                                             | Any editor; github.com                                                          |

## Phase 1: Read the law

### Step 1. Fix the scope and the as-of date

Decide exactly which provisions the ontology covers, and as of when. Write both into the ontology header and the README before you write a single class.

- Name the chapters and subchapters. TX-DNR covers Health & Safety Code chapter 166, subchapter C (out-of-hospital orders) and subchapter E (facility orders), and nothing else.
- Read the **history note** at the foot of each section. It tells you which acts amended it and when. Section 166.203 was amended in 2023, which added a third route to a valid facility order; an older copy of the law has only two.
- Never work from a bill's enrolled text alone. It shows the law as passed, not as later amended. Use it for history; use the current code for content.

## Step 2. Read every section and build a section table

Read the definitions section first, then every operative section, and record each rule in a working table. One row per rule:

| Section    | Who                                            | Must, may, or may not | Do what                        | Conditions                                                    | Exceptions                                                    |
|------------|------------------------------------------------|-----------------------|--------------------------------|---------------------------------------------------------------|---------------------------------------------------------------|
| 166.089(a) | responding EMS personnel                       | must                  | honor an out-of-hospital order | form found on arrival; identity confirmed; form appears valid | 166.089(h) request at scene; 166.098 known pregnancy          |
| 166.092(a) | declarant, guardian, qualified relative, agent | may                   | revoke the order               | at any time; declarant regardless of competence               | oral revocation effective only once communicated at the scene |

- Follow every cross-reference, and read the section it points to. Section 166.088 depends on 166.039(b) for the order of qualified relatives; 166.082 depends on 166.003 for witness qualifications.
- Quote exact words for conditions. "Appears to be valid" and "is valid" are different rules, and the difference decides scenario S5 later.
- Mark every place the statute says what does *not* follow. Section 166.204(b) says a missing notice "does not affect the validity" of an order. Rules like that are easy to miss and change outcomes.

## Step 3. Sort every noun into a Foundry pattern

Go down the section table and ask of each thing the law mentions what kind of thing it is. The answer picks its parent class. [Overview - Legal Ontologies Foundry](#)

| If it is...                                       | It is a...    | Parent                | TX-DNR examples                              |
|---------------------------------------------------|---------------|-----------------------|----------------------------------------------|
| What the law or an order says, surviving any copy | legal content | ODP-001 legal content | out-of-hospital DNR order, advance directive |

| If it is...                                                    | It is a...     | Parent                                  | TX-DNR examples                                     |
|----------------------------------------------------------------|----------------|-----------------------------------------|-----------------------------------------------------|
| A paper, file, or object carrying that content                 | legal document | ODP-001 legal document                  | order form; identification device (material entity) |
| Something people do that makes, changes, or ends legal content | legal act      | ODP-001 legal act                       | execution, issuance, revocation                     |
| Something a person is, because of law or appointment           | legal role     | ODP-002 legal role                      | declarant, attending physician, qualified relative  |
| A must, may, can, or cannot of a person                        | legal position | ODP-003 duty, claim, power, immunity... | duty to honor, power to revoke                      |
| A happening that is not itself a legal act                     | process        | BFO process                             | EMS response, CPR, pregnancy                        |

When something fits two rows, it is usually two things. "The order" is the content (what it directs), the form (the paper), and the execution (the signing). Model all three and relate them; ODP-001 exists for exactly this.

**Optional: let bfo-agent draft the first pass.** Give bfo-agent the text of the in-scope sections and let it propose classes and relations. Its anti-pattern rules keep the draft clean in ways a hurried human often is not: no privation classes ("non-revoked order"), no boolean expressions minted as IRIs, no propositions reified as classes, no re-minted BFO identifiers. Its OwlTester gate means the draft arrives consistent and BFO-grounded. Treat it as raw material, not as the ontology: it mints its own vocabulary under BFO directly, so step 5 re-parents it onto the Foundry patterns and checks every term against your section table.

## Phase 2: Build

### Step 4. Set up the repository

1. Choose a registry identifier and prefix (TX-DNR used `txdnr` and `TXDNR`), and reserve your ID range: you mint `TXDNR_0001000` to `TXDNR_0001999`.
2. Create the folders: `src/ontology` (editors' file and imports), `src/examples` (scenarios), `src/sparql/qc` (checks), `tools` (report script), `docs`.
3. Put BFO 2020 core and the merged LOF-ODP-001 to 004 base files in `src/ontology/imports`, each re-annotated with an import IRI under your namespace (`robot annotate --ontology-iri ...`).
4. Write a `catalog-v001.xml` that maps each import IRI to its local file, so Protégé and ROBOT work offline.
5. In Protégé, set **Preferences > New Entities** to mint IRIs with your prefix, seven digits, from the start of your range, and to label new entities with `rdfs:label`.

## Step 5. Write the terms

Work through the step 3 table, one class at a time. For each:

1. **Label:** lower case, singular, unique. Add a qualifier when a label would collide with an imported term ("Texas DNR order", not "DNR order").
2. **Parent:** the class step 3 chose.
3. **Definition** (`skos:definition`): genus and difference, in one sentence, using the statute's own words where you can. "An out-of-hospital DNR order execution in which a competent person signs the order for themselves, before two witnesses or with the signature acknowledged before a notary public, and the attending physician signs it."
4. **Source** (`dcterms:source`): the exact section, to the subsection. Every legal class gets one.
5. **Jurisdiction** (`dcterms:coverage` "Texas"): on every class specific to the jurisdiction. Generic classes (CPR, defibrillation) do not get it. This is Foundry principle P-010.
6. **Comment** (`rdfs:comment`): the exceptions and limits the definition cannot carry, quoted. "Does not authorize withholding comfort care" goes here.
7. **Axioms:** only what the statute fixes. An order form is a material entity that *is carrier of some* out-of-hospital order; the three facility-order routes are disjoint.

If you started from a bfo-agent draft, revise it rather than accept it:

1. **Re-parent** each class from its BFO parent to the matching Foundry class from step 3. A drafted *DNR order* under *generically dependent continuant* becomes a subclass of ODP-001 *legal content*; a drafted *declarant* under *role* becomes a subclass of ODP-002 *legal role*.
2. **Merge duplicates and split conflation.** Look especially for one class doing the work of three, the order, its form, and its execution.
3. **Re-mint identifiers** into your reserved range, so released IRIs follow the Foundry's identifier policy.
4. **Add what extraction rarely supplies:** the `dcterms:source` section on every legal class, `dcterms:coverage`, and the exceptions in `rdfs:comment`.
5. **Delete anything with no row in the section table.** An extracted term the statute does not support is a claim the ontology cannot back.

Relations get the same treatment: label, definition, source, domain, and range.

One technique worth knowing: when legal content names a *kind* of thing rather than a particular one (an order "directs withholding of" CPR as a kind, not one episode of CPR), declare the kind as both a class and an individual. OWL 2 allows this (punning), and it lets a query ask which treatments an order covers.

**Try a hard axiom in OwlTester first.** Before you commit an axiom you are unsure of, write it as a first-order expression and paste it into **Analysis Tools > Test FOL Expression**. The tool accepts BFO-style `instance_of(x, Class, t)` syntax and reports the BFO classes and relations it recognized and any non-BFO terms. A misspelled BFO relation, or a term you meant to import but did not, shows up under *Non-BFO Terms* before it reaches the ontology. For example, the rule that every out-of-hospital order form carries an order:

```
forall x t (instance_of(x, OutOfHospitalDNROrderForm, t) ->
  exists y (instance_of(y, OutOfHospitalDNROrder, t) & is_carrier_of(x, y, t)))
```

The **FOL Syntax Guide** and **Prover9 & CLIF Guide** under the same menu give the accepted forms.

## Step 6. Run the checks, and fix until clean

Run these after every working session, not just at the end:

```
robot merge --catalog catalog-v001.xml --input lof-txdnr-edit.ttl \
  reason --reasoner HerMiT --equivalent-classes-allowed asserted-only \
  verify --queries ../sparql/qc/*.sparql --output-dir tmp
robot merge --catalog catalog-v001.xml --input lof-txdnr-edit.ttl \
  report --fail-on ERROR --output tmp/report.tsv
```

The first command fails if the ontology is inconsistent or any class is unsatisfiable, then runs four SPARQL checks: every term has a label, a definition, and a BFO ancestor, and no two terms share a label. The second runs ROBOT's standard quality report. A Makefile that runs both as `make test` saves retyping. In Protégé, the same reasoner check is **Reasoner > HerMiT > Start reasoner**.

**Then get a second opinion from OwlTester.** Upload the full release file (step 11's `lof-<id>.owl`, merged and reasoned) at **Analysis Tools > Upload Ontology**. Its report gives, in one place, the consistency and coherence verdicts, any class placed under two BFO categories that BFO declares disjoint, an entity inventory, and a Prover9 and Mace4 cross-check of the first-order translation. **History** keeps every run, so you can compare versions.

Read the report knowing what it does and does not test, as of the October 2026 runs on the LOF patterns:

- Its BFO check is a category-disjointness lint. It catches a class placed under both *process* and *continuant*, not a class placed under the wrong one.
- Its first-order cross-check covers subsumption and disjointness axioms. Existential restrictions (*every form carries some order*) were not carried into it, and the prover ran without BFO's own axioms loaded (the report's "bfo\_background": false). Agreement there confirms the class hierarchy, not the restrictions.
- Terms appear by IRI, not label, so keep your label list open beside it.

Use ROBOT as the gate that must pass, and OwlTester as the independent check that should agree. A disagreement between them is worth investigating before release.

## Phase 3: Test against cases

### Step 7. Write scenarios, one rule each

A scenario is a small set of hypothetical people and events, written as instance data in `src/examples/scenarios.ttl`. Its purpose is to test one rule.

- **Start from the section table.** For each rule with a condition or exception, write one scenario where it applies and change one fact to make another where it fails. TX-DNR's S1 (valid order, honored) and S2 (same, but a relative asks for CPR) differ in a single line.
- **Write the law itself once**, at the top: the Legislature, its power, the enactment, and chapter 166 as legal content. Every duty can then trace back to it.

- **Build one full Minimum Legal Chain** (LOF-ODP-004) for at least one duty: grounded in chapter 166, triggered by an occurrence, generated by an act realizing a role, owed to someone, with a remedy. If the reasoner classifies it as a *fully grounded legal position*, your classes connect correctly to the Foundry patterns.
- **Use hypothetical names**, and say so in the file.

After writing them, reason over the scenarios and check that the inferences you expect appear. TX-DNR expected, and got, the five treatments each out-of-hospital order covers, filled in from the class axioms without being asserted.

### Step 8. Write the rules that decide

OWL alone cannot say whether responders must honor an order, because OWL assumes the world is open: it never concludes that a missing fact is false, so it can never find that no one asked for CPR. Decisions therefore go in rules that check what is recorded.

- Write one rule per statutory condition, as a SPARQL ASK query, paired with its section and a sentence saying what is the case when it fails.
- **Keep three kinds of rule apart.** *Honoring rules* decide what responders must do. *Validity rules* decide whether an order was properly made. *Duty rules* record obligations whose breach does not void the order. Mixing the last two was the most consequential mistake in the first TX-DNR draft.
- Put the rules in a short script (`tools/txdnr_report.py`) that runs them over the reasoned scenarios.

### Step 9. Read the report as a lawyer would

Run `make report` and read every verdict against the section table:

- Is each verdict what the statute requires, for the reason the statute gives?
- Does each failure message say what *is* the case, without double negatives? "The responders know the person to be pregnant" reads; "Not met: the responders do not know the person to be pregnant" does not.
- Where two rules interact, is the interaction right? In S5 the form appears valid, so responders must honor it, even though the order was improperly made for a minor. The report says both, and why.

## Phase 4: Check and publish

### Step 10. Get an independent statutory review

The builder is the worst person to find the builder's mistakes. Give a second reader who has not seen the work the editors' file, the rules script, and one instruction: check every legal claim against the current statute, section by section.

If bfo-agent drafted the ontology, the reviewer must be a person or a separate agent session with fresh instructions, never the session that did the drafting: a reviewer that shares the drafter's reading of the statute will confirm it. TX-DNR's review was done by a separate agent session given only the two files

and the instruction above, which fetched every cited section itself. OwlTester cannot do this step. It checks logic against BFO, not terms against the law.

- Ask for one verdict per claim: **correct**, **wrong** (with what the statute actually says), **miscited** (right rule, wrong section), or **unverifiable**.
- Ask, separately, for important rules the ontology **omits** entirely.
- The reviewer must read the sections themselves, not your summary of them.
- **Spot-check before you fix.** Re-read the statute for any finding that would change a verdict. In TX-DNR, the two outcome-changing findings (identification devices, and notice not affecting validity) were each confirmed against the section text before anything was changed.
- **Fix, then rerun everything** from step 6. A fix in the ontology can break a rule, and a new rule needs a new scenario. TX-DNR added two scenarios (S9 and S10) to test rules the review revealed.

What the review finds and you do not fix goes in the README's *not yet modeled* list, with sections.

## Step 11. Document and package

The README is where a reader decides whether to trust the ontology. It should carry:

- **Scope and as-of date**, and the jurisdiction.
- **A statute map**: each section, the rule, and how it is modeled. This is your section table from step 2, now with a modeling column.
- **The scenarios and their verdicts**, in a table.
- **What is not yet modeled**, with sections.
- **Sources**, linked, and a plain statement that it is a model, not legal advice.
- A LICENSE file with the full CC BY 4.0 legal code, so GitHub recognizes it.
- The Foundry **registry entry** (ontology/<id>.md), checked against the registry schema before you submit it.

Then run `make release` to write the two release files: the full ontology (merged with imports and reasoned) and the base (your own axioms only, which others import).

**Record provenance.** Add a short *How this was built* note to the README: which terms bfo-agent drafted and which were written by hand, the OwlTester analysis numbers for the released files, and who did the statutory review and when. A reader deciding whether to trust a legal ontology needs to know which parts a person checked against the statute.

## Step 12. Publish

1. Create the repository in the Foundry organization as `lof-<id>`, and upload the files.
2. Add `.github/workflows/qc.yml` so the checks from step 6 run on every change.
3. Tag the release `v0.1.0` and mark it a pre-release.
4. Open a pull request on the Foundry site adding the registry entry at draft status. Under the governance rules, one co-founder's approval suffices.
5. Open an issue for each open question and each *not yet modeled* item, so reviewers can take them up one at a time.

## Mistakes the review caught

The first TX-DNR draft passed every automated check and still got the law wrong in five places. The reasoner checks that the ontology is coherent, not that it is faithful to the statute; only a reader with the statute open can check that.

| What the first draft did                                                        | What the statute says                                                                | Lesson                                                                     |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Required responders to find the order form                                      | A worn DNR identification device must be honored as the form would be (§ 166.090(b)) | Read the sections a rule cross-references, not only the rule itself        |
| Treated a missing notice as making a hospital order invalid                     | Failure to give notice "does not affect the validity" of the order (§ 166.204(b))    | Separate validity rules from duty rules (step 8)                           |
| Gave only the declarant the power to revoke                                     | A guardian, qualified relative, or agent may also revoke (§ 166.092(a))              | Model who holds a power from the statute's list, not from the obvious case |
| Cited one immunity section for both kinds of order                              | Out-of-hospital orders: § 166.094; facility orders: § 166.207, with different terms  | Check each citation at the subsection level                                |
| Treated an advance directive as a directive or a medical power of attorney only | An out-of-hospital DNR order is also an advance directive (§ 166.002(1)(B))          | Reread the definitions section after modeling, not only before             |

Two habits prevent most of these: quote the statute's words in definitions and comments rather than paraphrasing, and keep the section table open beside the editors' file the whole time.